

Sanity Harness: the lim section

Y. Xiang

1 Stub Toolkit Section

Definition 1.1. *stub*

Definition 1.2. *stub*

Definition 1.3. *stub*

Lemma 1.1. *stub*

Theorem 1.1. *stub*

2 Stub Limits Section

Proposition 2.1. *stub*

Lemma 2.1. *stub*

Theorem 2.1. *stub*

3 Stub Variants Section

Definition 3.1. *stub*

Theorem 3.1. *stub*

Theorem 3.2. *stub*

Corollary 3.1. *stub*

Conjecture 3.1. *stub*

4 Stub Recursion Section

Theorem 4.1. *stub*

Theorem 4.2. *stub*

Proposition 4.1. *stub*

Definition 4.1. *stub*

Theorem 4.3. *stub*

Remark. *stub*

Theorem 4.4. *stub*

Corollary 4.1. *stub*

4.1 Converging to the Fixed Point

Restart iterates a *rule*; we now iterate an *expression*. The variant of this subsection adds one node to the grammar: apply the unary expression to its own output until nothing changes. Where Definition 3.1 normalizes one rewrite rule $B \rightarrow A$ under the leftmost strategy, the new node normalizes a whole pipeline under the strategy the calculus already has – the sweep – and the two normalizers of the same rule will turn out to be genuinely different functions (Remark 4.1 below). The reward for the difference is a characterization the restart calculus does not have: with this one node the flat calculus computes *exactly* the partial computable functions, and the entire distance between the bounded calculus of Theorem 2.1 and universality turns out to be one iteration node (Corollary 4.2).

Definition 4.2. (Fixed-Point Iteration) *Extend the grammar of Definition 1.1 by the rule*

$$\frac{E \in \text{Exp}_1, E_0 \in \text{Exp}_n}{\lim(E)[E_0] \in \text{Exp}_n}$$

where X_1 is bound in E by the node; the free variables of $\lim(E)[E_0]$ are those of E_0 . The orbit of $E \in \text{Exp}_1$ on w is $s_0 = w$ and $s_{k+1} = \llbracket E \rrbracket(s_k)$, and

$$\llbracket \lim(E)[E_0] \rrbracket(\mathbf{S}) = s_K \quad \text{for the least } K \text{ with } s_{K+1} = s_K, \quad (1)$$

with $s_0 = \llbracket E_0 \rrbracket(\mathbf{S})$: the first state the iterand does not change. If some s_{k+1} is undefined, or no such K exists, the node is undefined (in the second case it diverges). Substitution acts on the start expression,

$$\lim(E)[E_0][F_1/X_1, \dots, F_n/X_n] = \lim(E)[E_0[F_1/X_1, \dots, F_n/X_n]], \quad (2)$$

capture-free, since only X_1 is bound and only in E – so Lemma 1.1 survives verbatim for the extended grammar.

An $L+\text{lim}$ expression is *flat* if its iterands contain no $\lim$ nodes; a single node $\lim([A/B])$ iterates one constant pass, the *sweep* $w \mapsto [A/B]w$. The preceding remark's sort has its sweep twin: $\lim([ba/ab])$ also maps every input to $b^{\#b}a^{\#a}$ (verified on all binary inputs of length ≤ 12 and 100 random ones of length 13–16).

Lemma 4.1. (The Fixed Points of a Pass) *For $B \neq \epsilon$ and $A \neq B$:*

$$[A/B]w = w \iff B \not\subset w. \quad (3)$$

Proof. If B does not occur, the scan never fires. If it does, the sweep performs $m \geq 1$ non-overlapping replacements, changing the length by $m(|A| - |B|)$ – so w is changed whenever $|A| \neq |B|$ – and when $|A| = |B|$ (with $A \neq B$) the first replacement spells A where w spells B at the same position, with the prefix before it untouched. $\square$

So for $A \neq B$ the sweep's fixed points are exactly the B -free strings – the same stop set as the restart process of Definition 3.1, which stops at the first B -free trajectory term. Lemma 4.1 is also a complete decision procedure for the *occurrence* of a constant, and at one stroke a subroutine of the compilation below: a pass with a one-character replacement tests containment,

$$\text{contains}_B(E) \triangleq \text{eq}([c/B]E, E), \quad c \text{ any character with } \{c\} \neq B, \quad (4)$$

because $[c/B]E = E$ iff $B \not\subset E$.

Proposition 4.2. (One-Rule Totality) *Let $B \neq \epsilon$.*

- (i) $|A| \leq |B|$ and $A \neq B$: $\lim([A/B])$ is total.
- (ii) $B \subset A$: $\lim([A/B])$ diverges on every B -containing input.
- (iii) $A = B$: $\lim([A/B])$ is the identity.

Proof. The proofs of Theorem 3.1(i) and (ii) bound a replacement at *any* position – length in (i), the potential V in (ii) – so each sweep with at least one replacement strictly decreases a measure that takes finitely many values, and the orbit of sweeps reaches a B -free string, a fixed point by Lemma 4.1: (i). For (ii), every replaced site re-contains B , so after every sweep the string still contains B and is not a fixed point; for (iii) the pass is the identity, so the first sweep already is a fixed point. Note the one divergence from the restart column: at $A = B$ the restart process still hunts a redex and diverges (Theorem 3.1(iii)), while the sweep stops – the two stop rules agree exactly when $A \neq B$. $\square$

Remark. (Two Normalizers, Neither Canonical) $\lim([A/B])$ and $[A/B]^m$ are two deterministic normalizers of the one-rule system $B \rightarrow A$, and one-rule systems are not confluent: they disagree. On the census domain of Theorem 3.1(v) – the 930 binary rules with $|A|, |B| \leq 4$ on all inputs of length ≤ 9 , restart at a $3 \cdot 10^5$ step cap – the two disagree on *value* on 3,306 pairs and on *termination* on 290 pairs, the latter exactly the two rules $[abba/bab]$ and $[baab/aba]$, all in the direction restart-diverges, sweep-converges, and *none* in the reverse direction; re-verified on all inputs of length ≤ 6 at a 2,000 step cap (126 value, 20 termination disagreements, 0 reverse). Witnesses, at a 10^5 step restart cap: $[ab/bb]$ on $bbbb$ gives $abab$ (one sweep replaces the two disjoint occurrences) against restart's $aaab$ (three sequential leftmost splices, each shifting the next site); $[ab/abba]$ on $abbababba$ gives abb in two sweeps against restart's $abbba$; and $[baab/aba]$ on $aaaba$ goes $aaaba \rightarrow aabaab \rightarrow abaabab \rightarrow baabbaabb$ – three sweeps, done – where restart diverges, each leftmost splice re-creating an aba at the junction, the census mechanism of Theorem 3.1(v). The sweep is thus *sometimes* the better-defined of the two strategies, and sometimes not definable from it at all: neither normalizer computes the other's values on a non-confluent rule.

The rest of this subsection shows that this modest operator – iterate a flat expression to its fixed point – is exactly the missing piece between the polynomial calculus of Theorem 2.1 and universality.

Theorem 4.5. (Compiling Two-Counter Machines Flat) *Fix Σ with $|\Sigma| \geq 2$ and two characters $\sigma \neq \tau \in \Sigma$. For every two-counter machine M with instructions $\text{inc}(r, j)$, $\text{decjz}(r, j_0, j_1)$ and halt (counters $r \in \{1, 2\}$), there are flat expressions $\text{INIT}, \text{step}, \text{OUT} \in \text{Exp}_1$ such that*

$$\text{MAIN} \triangleq \text{OUT}[F/X_1], \quad F \triangleq \text{lim}(\text{step})[\text{INIT}], \quad (5)$$

has exactly one lim node and computes the partial function of M on tallies: $\text{MAIN}(\tau^n)$ is the counter-2 tally $\tau^{f(n)}$ when M halts on input n with counter 2 equal to $f(n)$ and counter 1 empty, and is undefined exactly when M does not halt. The iterand step is an if-chain of the Section 1 constructions over constant occurrence tests (4), all of whose branches are constant passes.

Proof (Proof (construction)). Three compile-time transformations of M come first, each preserving its partial function: (N1) a decjz whose zero-branch jumps to itself would leave the configuration untouched – a non-halting fixed point of the compiled step – so it is replaced by a fresh self-looping increment (both diverge); (N2) the output is read from counter 2 (swap the counters if M leaves its result in counter 1); (N3) jumps to the halt instruction are retargeted through a fresh draining instruction $\text{decjz}(1, \text{halt}, \text{drain})$, so counter 1 is empty at halt.

Frames. Write $V(k) = \sigma^2(\tau\sigma)^{k-1}\tau\sigma^3$ for $k \geq 1$: k isolated τ 's, head run σ^2 , interior σ -runs of length 1, tail run σ^3 , length $2k + 4$. The compiler assigns each frame constant a distinct $V(k)$ with $k \geq 2$: delimiters D_0, D_1, D_2, D_3 , instruction codes $P_1, \dots, P_s, P_H$ for the instructions of the normalized machine, and the wrap marker M .

Configurations. For $i \in \{1, \dots, s\}$ let $\Pi(i)$ be the program list $P_1 \dots P_s P_H$ with the i -th code wrapped, and $\Pi(0) = P_1 \dots P_s M P_H M$ the halted list. The configuration with instruction counter i and counter values x, y is

$$C(i, x, y) = D_0 \Pi(i) D_1 \tau^x D_2 \tau^y D_3, \quad (6)$$

an empty tally meaning adjacent delimiters.

The step. On a configuration the current instruction is the wrapped one, so dispatch is the occurrence test of $M P_i M$ (4): step is the if-chain $\text{if}(\text{contains}_{M P_i M}, \text{body}_i, \dots)$ over $i = 1, \dots, s$, with the identity as the final else – the halted configuration is a fixed point. Each body_i is three constant passes:

- (a) *operate*: for $\text{inc}(r, j)$, the pass $[D_r \tau / D_r]$ (a new tally character at the left edge of counter r 's run); for $\text{decjz}(r, j_0, j_1)$, the test $\text{contains}_{D_r D_{r'}}$ – adjacent delimiters, i.e. counter r empty (Lemma 4.2(iii)) – selects between the jump j_0 and $[D_r / D_r \tau]$ followed by the jump j_1 (the decrement deletes the first tally character, pinned by Lemma 4.2(iv));
- (b) *unwrap*: $[P_i / M P_i M]$;
- (c) *wrap*: $[M P_j M / P_j]$ for the jump target j .

Unwrap before wrap: on a self-jump ($j = i$) the wrap pattern P_j would otherwise occur inside the block being unwrapped, and double-wrap. Every pattern is a

nonempty constant, so every branch is total on every input, and an unselected branch's passes are merely *discarded*: nothing needs the lazy gate of Section 4, because a flat branch has no undefined part to guard. (For the doubling machine: step is 298 nodes, 83 passes, of which 75 constant-pattern; MAIN is 310 nodes.) $\square$

Lemma 4.2. (Occurrence) *Let the frame constants be assigned pairwise distinct family indices $k \geq 2$. Then for every instruction counter $i \in \{0, \dots, s\}$ and every $x, y \geq 0$, in $C(i, x, y)$:*

- (i) *every frame constant occurs exactly at its slots (M twice, at the two wrap positions; every other constant once);*
- (ii) *MP_iM occurs iff i is the current instruction ($i \geq 1$), and MP_HM iff $i = 0$; in either case exactly once;*
- (iii) *$D_rD_{r'}$ occurs iff counter r is zero, exactly once;*
- (iv) *$D_r\tau$ occurs iff counter r is nonzero, exactly once, ending at the first tally character;*
- (v) *the output prefix $D_0\pi(0)D_1D_2$ occurs iff $i = 0$ and $x = 0$, exactly once, at position 0.*

Proof. The proof is a *gap calculus*. For a string W containing at least one τ and no $\tau\tau$, let its *gap sequence* be $(h; g_1, \dots, g_{m-1}; t)$: the number of σ 's before the first τ , between consecutive τ 's, and after the last.

Step 1 (configurations). Every frame constant has gap sequence $(2; 1^{k-1}; 3)$. Concatenating two constants merges the trailing σ^3 of the first with the leading σ^2 of the second: an interior gap of 5. A tally τ^n , $n \geq 1$, between its delimiters contributes $3 \cdot 0^{n-1} 2$; an empty tally makes them touch, a gap of 5. So a configuration's gaps are: head 2, then the constants' interior 1-blocks interleaved with junction gaps of 5, the two tally junctions $3 \cdot 0^{x-1} 2$ or 5, tail 3.

Step 2 (patterns cannot cross tallies). Every pattern of the construction has at least two τ 's, no $\tau\tau$, head exactly 2, tail in $\{0, 3\}$, and interior gaps in $\{1, 3, 5\}$ only – never 0 and never 2. If a pattern W occurs in $C(i, x, y)$, its τ 's map to *consecutive* τ 's of the configuration (a skipped τ would have to appear among W 's σ 's), so every interior gap of W equals the corresponding gap of the configuration. Gaps of 0 arise only inside tallies of length ≥ 2 , so W covers no two consecutive tally characters; a gap of 2 arises only after the last character of a tally, so W 's τ -sequence cannot cross a tally's right edge; and if W 's first τ were a tally character, its next gap would have to be 0 or 2 – both impossible. The only pattern that can meet a tally at all is the decrement pattern $D_r\tau$, whose last gap 3 pins its last τ to the tally's *first* character. Every other pattern avoids tallies entirely.

Step 3 (pinning). Let W 's leading segment (up to its first gap of 3 or 5) contain k τ 's, all separated by gaps of 1. Its first τ must be a constant's *first* τ : a middle τ of a constant has one σ before it inside the constant, and W 's head σ^2 must fit in the preceding σ -run. If that constant had more than k τ 's, the $(k+1)$ -st would follow a gap of 1 where W has its seam 3 or 5: impossible; so the constant has exactly k τ 's, and by the distinctness of family indices it

is the unique slot holding that string. The same argument applies constant-by-constant along W 's seams (a seam of 5 matches a junction, so the constants on both sides occupy adjacent slots). Assembling: MP_iM can only chain the M -slot, a junction, the P_i -slot, a junction, the M -slot, and the two M -slots of a configuration bracket exactly one code – the wrapped one: (ii). $D_rD_{r'}$ chains D_r 's slot to the next only through a gap of 5, which after D_r 's slot arises exactly when the tally is empty: (iii). The output prefix is pinned at D_0 's slot and then slot-by-slot through the program zone (matching $\Pi(0)$ iff $i = 0$), then across the first tally junction (5 iff $x = 0$): (v). Finally a single frame constant $V(k)$ is itself such a pattern, pinned by Step 3 to a slot holding that string: the two M -slots are the two wrap positions, every other constant has one slot: (i). $\square$

Proof (Proof of Theorem 4.5 (correctness)). $\text{INIT} = D_0 \Pi(1) D_1 \cdot [\tau/\sigma] X_1 \cdot D_2 D_3$ maps any w over $\{\sigma, \tau\}$ to $C(1, |w|, 0)$: one constant pass turns the input into a τ -tally of its length, and the frame constants sandwich it. By Lemma 4.2 the dispatch test of the current instruction is the only one that fires, the operation pass acts on exactly the intended tally, and unwrap-then-wrap moves the wrap to exactly the target code – so an induction along the run of M gives $\text{step}(C(i, x, y)) = C(i', x', y')$ for the machine's step, and the halted configuration – where every dispatch test fails and the identity branch runs – is a fixed point. It is the unique fixed point among configurations: every non-halted configuration moves, since its instruction either edits a tally or jumps (the zero-branch self-jump that would not move is excluded by (N1)). The orbit of $\lim(\text{step})$ on a configuration is therefore exactly the run of M , sweep for step: it converges iff M halts, in as many sweeps as the machine takes steps. At halt, (N2) puts the result in counter 2 and (N3) empties counter 1, so Lemma 4.2(v) applies to the configuration $C(0, 0, y)$: $\text{OUT} = [\epsilon/D_0 \Pi(0) D_1 D_2][\epsilon/D_3]$ deletes the constant prefix (which occurs exactly once, at position 0) and the closing delimiter, leaving τ^y . Both deletion passes sit outside the iterand, so step itself is total.

Verified end to end (the scripts and full domains are in the research log, research/scratch/lim/): the compiled doubling machine on $n \leq 12$ (τ^{2^n} exact), the adder on all 25 input pairs $n, m \leq 4$ plus a 7×7 border grid, a counter-draining machine on a 5×5 grid, and the normalization pipeline (N1)–(N3) demonstrated on a machine needing all three, with the raw defects shown (a non-halting fixed point, a counter-1 output, a dirty halt) and repaired; divergence verified on a self-looping and a two-cycling machine, the orbit matching the simulated run on every point; step's totality on 400 random large strings and on every branch; the occurrence census of Lemma 4.2 at the edge lengths $x, y \in \{0, 1, 2\}$ for every instruction counter, systematically for $x, y \leq 40$, and on 400 random configurations up to 1000, at three constant assignments (a fourth, violating the distinctness hypothesis, run as a stress test); and the whole of it re-run under both swaps of the Section-2 toolkit roles. $\square$

Theorem 4.6. (The Compiled Program Computes the Machine) *Over any alphabet with $|\Sigma| \geq 2$, the flat calculus with one lim node computes exactly the partial computable functions $\Sigma^* \rightarrow \Sigma^*$: it computes all of them (Theorem 4.5*

and Minsky's two-counter machines [1], arbitrary inputs brought to tallies by the two-letter prefix-code reduction of Theorem 4.4), and it computes nothing else: a converging evaluation is a finite sequence of sweeps, each a primitive-recursive pass, halted by a decidable fixed-point test, so every denotation is partial computable.

Proposition 4.3. (One Node Is Enough: the Driver) *Every $L+\text{lim}$ expression is computed by a lazy-pass recursive L expression of the same size up to a constant factor: the unary function $\text{lim}(E)$ over a flat iterand E is the guarded definition*

$$\text{RUN}(X) \triangleq \text{if}(\text{eq}(E(X), X), X, \text{RUN}(E(X))), \quad (7)$$

with the recursive call in the dead branch's replacement – the pattern-gated position of Remark 4, where the two-way gate of Theorem 4.3 makes *if* sound both ways.

Proof. Induction on the expression, replacing each *lim* node by its driver bottom-up: the iterand E is then *lim*-free by the induction hypothesis, and the node $\text{lim}(E)[E_0]$ is the driver applied to the start, $\text{RUN}[E_0/X]$. The driver unrolls the orbit exactly: its guard tests $s_{k+1} = s_k$, and its else-branch forces $E(X)$ – one sweep – before recurring, so the recursion halts on the least K with $s_{K+1} = s_K$ and diverges with the orbit; and if $E(X)$ is undefined at any point, the guard is undefined and so is the driver, exactly as the orbit's undefinedness makes the node undefined (Definition 4.2) – the two are undefined together, as in Lemma 1.1's caveat. What the eager calculus could not do – evaluate a branch that may diverge – is not asked of it: the dead branch's round is a discarded replacement (Remark 4's minimal repair), and the two-way gate of Theorem 4.3 is what makes that sound. Verified: the driver against the direct orbit for eight rules (convergent and divergent, census witnesses included) on 19 inputs each under both choices of the toolkit roles – 135 convergent orbits value-exact, 29 divergent – plus the compiled iterands of Theorem 4.5 (the drivers of the doubling and adder machines match MAIN exactly on $n \leq 5$) and a self-looping machine diverging on both sides. $\square$

Corollary 4.2. (One Node Is the Whole Gap) *Over any alphabet with $|\Sigma| \geq 2$:*

$$\begin{aligned} \text{flat } L + \text{ one lim node } &= \text{ lazy-pass recursive } L \\ &= \text{ the partial computable functions. } \end{aligned} \quad (8)$$

Proof. Left to right by Proposition 4.3 (one driver per node; the calculus underneath is untouched), right to left by Theorem 4.6 together with the soundness direction of Theorem 4.4. $\square$

This is the inertness story of Section 4, sharpened to a single quantifier. The bounded calculus is polynomial (Theorem 2.1); recursion under eager evaluation changes nothing (Theorem 4.1), and neither do lazy arguments (Theorem 4.2);

the two-way gate plus recursion is universal (Theorem 4.4). The gate’s condition – evaluate the replacement only when the pattern fires – is exactly the fixed-point test of Definition 4.2, and Proposition 4.3 says the translation is an equality of classes, not an embedding of one into a richer one: the power was never in the recursion depth, it is unbounded guarded iteration, and one iteration node over a flat pipeline already has all of it.

Corollary 4.3. (Undecidability)

- (i) Given a flat $E \in \text{Exp}_1$ and w , whether $\lim(E)(w)$ is defined is Σ_1^0 -complete.
- (ii) Given a flat $E \in \text{Exp}_1$, whether $\lim(E)$ is total is Π_2^0 -complete.

Proof. Both problems are in their classes because “the orbit converges within k sweeps” is decidable, each sweep a primitive-recursive pass. Hardness: $\text{MAIN}(\tau^n)$ is defined iff M halts on n (Theorem 4.5), a Σ_1^0 -hard relation, and MAIN is total iff M halts on *every* input (every n is INIT ’s image of some string – every length is hit), a Π_2^0 -hard one. The undecidability of totality is the sibling of Corollary 4.1(4) for the recursive calculus. $\square$

Proposition 4.4. (Growth)

- (i) (exponential output, one node) Writing v for the Horner value of Theorem 3.2 (a adds 1, b doubles),

$$\lim([baa/ab])(w) = b^{\#b(w)} a^{v(w)} \quad (9)$$

for every w , in at most $v(w) - \#a(w)$ sweeps; in particular

$$\lim([baa/ab])(a b^{n-1}) = b^{n-1} a^{2^{n-1}} \text{ of length } 2^{n-1} + n - 1. \quad (10)$$

One constant pass under one $\lim$ node has exponential output: the length bound of Lemma 2.1 fails at depth one.

- (ii) (towers, nested nodes) The block $\lim([baa/ab]) \circ \lim([ab/aa])$ maps $b^m a^K$ to $b^{m+\lfloor K/2 \rfloor} a^{2^{\lfloor K/2 \rfloor + 1} - 2 + (K \bmod 2)}$ – the closed form of Corollary 3.1’s two restart nodes, now for sweep normalizers – so $2t - 1$ constant-pattern $\lim$ nodes produce output of tower height t in the input length, and $\lim(L) \sqsubseteq L$ fails: three nodes already outgrow Lemma 2.1’s polynomial bound.

Proof. (i) Each replacement $ab \rightarrow baa$ preserves $\#b$ and v : reading v left to right, $v(xab) = (v(x) + 1) \cdot 2 = v(xbaa)$, and appending the same suffix multiplies both equally – formally $v(uv) = v(u) \cdot 2^{\#b(v)} + v(v)$. So the sweep, which performs several such replacements at disjoint sites, preserves them too. The potential $\Phi(w) = v(w) - \#a(w)$ is nonnegative – v sums $2^{\#b(\text{after})}$ over the a ’s of w – drops by exactly 1 per replacement ($\#a$ grows, v does not), and is 0 exactly when no a has a b after it, i.e. on the ab -free strings, which by Lemma 4.1 are the sweep’s fixed points. The orbit therefore reaches an ab -free string after at most $\Phi(w)$ replacements – hence at most that many sweeps – and an ab -free string with the same $\#b$ and v is $b^{\#b} a^v$, both invariants forcing the exponents. The

potential also re-proves the termination of the restart process of Theorem 3.2, and the two normalizers of this rule agree on every input. (Verified: all binary inputs of length ≤ 12 , 200 random of length 13–16, the invariants per sweep on 50 traces, and the tally witness for $n \leq 14$.) (ii) The half node maps $b^m a^K$ to $b^m(ab)^{\lfloor K/2 \rfloor} a^{K \bmod 2}$ in *one* sweep – the greedy pairing of the a -run – and is then inert (ab contains no aa); composing with (i) and counting the b 's after each a of $(ab)^j$ gives the displayed form, the Horner computation of Corollary 3.1. (Verified against the closed forms for $t \leq 3 - 5$ nodes, output length 65,556 at $n = 4$ – and the block on the grid $m \leq 3$, $K \leq 14$.) $\square$

Remark. (Divergent Orbits Grow Like Fibonacci) The census's divergent rule $[aabb/ba]$ diverges under the sweep too, but with a law: on $aabbaabb$ the orbit lengths are $2F_{k+1} + 2k + 6$ for the Fibonacci numbers F , i.e. $8 \rightarrow 10 \rightarrow 14 \rightarrow 18 \rightarrow \dots \rightarrow 204$ in ten sweeps $\rightarrow \dots \rightarrow 1,028,520$ in twenty-eight – growth by the golden ratio per sweep, exponential in the iteration count; on $bbaa$ the law is $2F_{k+1} + 2k + 2$, and the mirror rule $[bbaa/ab]$ runs the reversed orbits (verified exactly through $k = 28$, the sweep strings cross-checked against the evaluator through $k = 16$). The mechanism is the census splice of Theorem 3.1(v) – a prefix ending aab against the replacement $aabb$, re-creating a junction ba – compounding: a junction in the context (b, a) doubles per sweep, a junction in (a, a) or (b, b) persists, one in (a, b) dies, and the two persistent species follow the Fibonacci recurrence. So the growth of the iterated calculus lives in the *iteration*, not the nesting: Corollary 3.1 needs $2t - 1$ nested nodes for tower height t , but one node already produces exponential output (Proposition 4.4(i)) and Fibonacci divergent orbits – while divergent restart rules were the *only* obstacle left in the one-rule census, here divergence is the engine.

The landscape closes one row and opens a question. The table of Section 3 gains:

variant	$L \sqsubseteq V$	$V \sqsubseteq L$	toolkit	growth
lim	trivially refuted	flat		unbounded

$L \sqsubseteq \lim(L)$ trivially, the grammar growing by one node kind; $\lim(L) \sqsubseteq L$ refuted by Proposition 4.4 (one node, exponential output); the toolkit carried verbatim – the compilation of Theorem 4.5 needs no recursion and no nesting, an if-chain of Section 1 passes; growth unbounded, the class being the partial computable functions. Restart and lim are the two iteration operators over the same grammar – one rule iterated against one expression iterated – and both escape L on growth (Corollary 3.1, Proposition 4.4), but lim's class is exactly characterized (Corollary 4.2) where the Markov row's entries carry Conjecture 3.1. What the characterization does *not* settle is the one-rule fragment: is convergence of $\lim([A/B])$ decidable? Proposition 4.2 settles everything outside the strategic case $|A| > |B|$ with $B \not\subseteq A$, and Remark 4.1 shows the sweep and leftmost versions of that case are genuinely different problems – the sweep sibling of Theorem 3.1(v)'s strategic termination problem, whose unrestricted form is the long-standing open one-rule termination problem [2]. And the hierarchy inside

the operator: one lim node over a *single constant pass* reaches exponential output and Fibonacci orbits (Proposition 4.4, Remark 4.1) – whether it reaches universality is open; the amplifier of Proposition 4.4(i) sorts and Horner-codes, and every fixed point of a one-rule sweep is B -free (Lemma 4.1), but no argument is known in either direction.

References

1. M. L. Minsky. *Computation: Finite and Infinite Machines*. Prentice-Hall, 1967.
2. A. Geser, J. Giesl. Termination of one-rule string rewriting systems. 2001.